

CSCI 135 Assignment – Tkinter TKalculator

November 20th, 2023

Due Date: Sunday, 12/03/2023 11:59PM

Points: 100

Topic: GUIs and Event-Driven Programming

In this assignment, you will be writing a GUI fraction calculator using Tkinter. In the prior two assignments you wrote and tested functions (`ExceptionalFractions.py`) and classes (`ClassyFractions.py`) to represent and manipulate fractions. In this assignment you will import and use one of these files to do the calculations for the fractions and operations selected and entered by the user of your GUI fraction calculator. Your GUI and associated code should be robust, and handle errors/exceptions in a graceful manner by displaying a message to the user within the interface rather than crash. (If there were errors in your `ExceptionalFractions.py` or `ClassyFractions.py` submissions, you should correct them.)

I am providing you with an example file `IntegerCalculator.py` that implements a simpler calculator to perform addition and subtraction of integers. You should find techniques used in this example sufficient to complete the more advanced GUI fraction calculator specified in this assignment.

Within this assignment you will find sample screens and input/output that you are to implement. There are multiple aspects to implementing this functionality: correct functionality, correct display and use of prior functions or classes. See later guidance for prioritization/importance of these aspects.

Make sure that you implement the functionality and display as best possible in your final `FractionCalculator.py` file and test it thoroughly.

Submission:

After following all instructions, please submit your Python scripts in Moodle. Upload these as individual files. Be sure the (main) file has an accurate header section including the file name, your name, credits, a description of the file, the class and other useful information such as example usage.

- `FractionCalculator.py`
- Your `ExceptionalFractions.py` or `ClassyFractions.py`, depending on which you used.

CSCI 135 Assignment – Tkinter TKcalculator

Grading:

This `FractionCalculator.py` script is worth 100 points. You will be graded according to the following criteria:

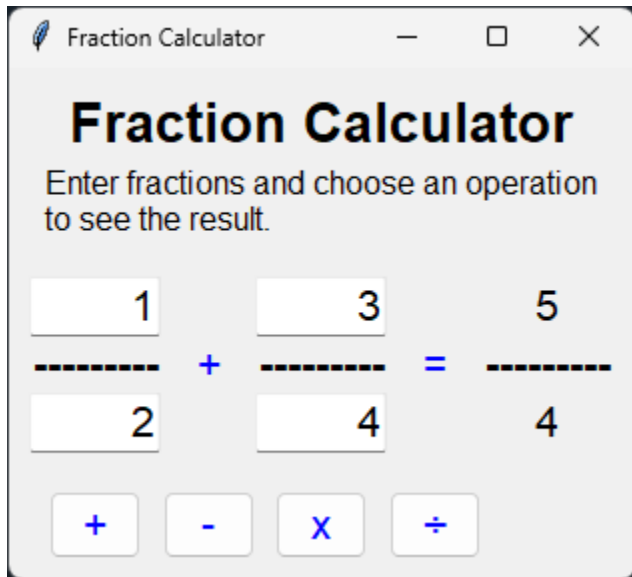
	Points
Header Section for File/Script	5
Header Comments for Functions	5
Descriptive Comments Throughout Code, Including Data	10
Functions Correctly – Matches what is shown in the assignment	20
Displays Correctly	20
Uses your <code>ExceptionalFractions.py</code> or <code>ClassyFraction.py</code>	20
Written Correctly – code written well and using constructs given	20
Total	100

FractionCalculator.py

The following sequence of screenshots demonstrates expected functionality and display to implement the fraction calculator GUI. Functionality and display should match as close as possible.

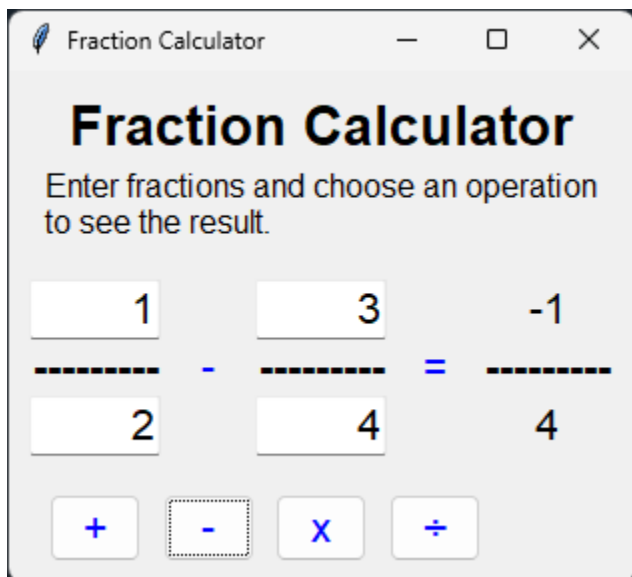
Startup

Upon startup, the display should look like this, including the same example data.



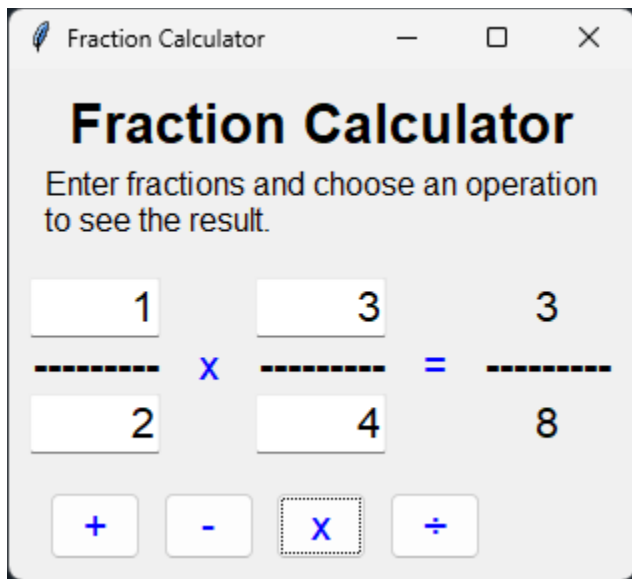
Press – button.

When the – (minus) button is pressed, the display should update to show the result.



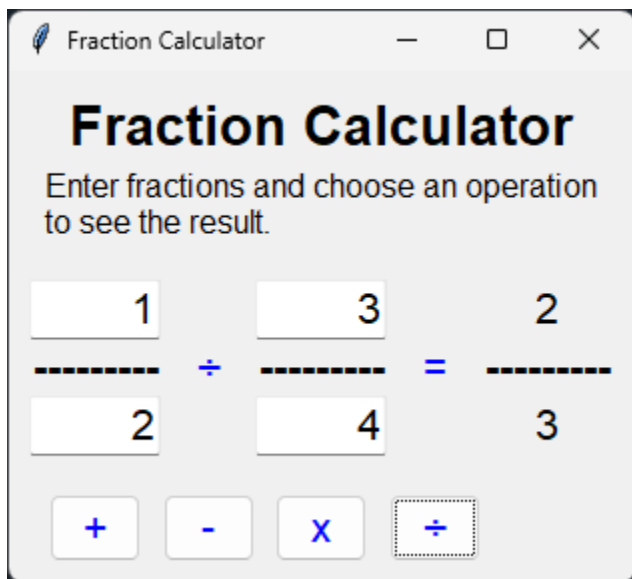
Press x button.

When the x (times) button is pressed, the display should update to show the result.



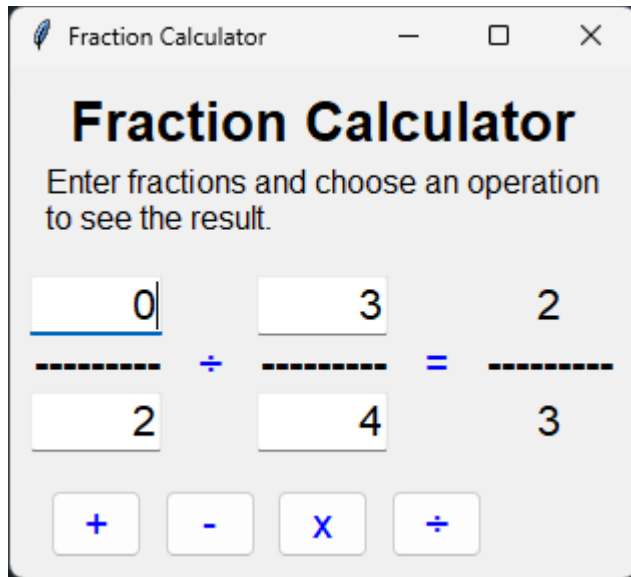
Press ÷ button.

When the ÷ (divide) button is pressed, the display should update to show the result.



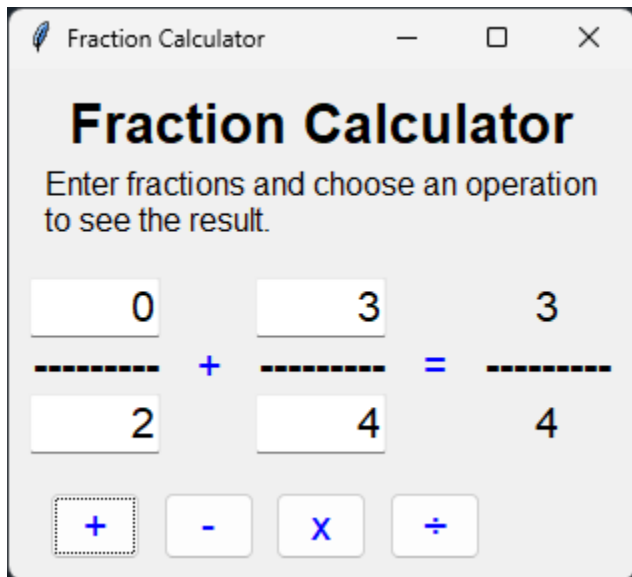
Change 1 to 0 in numerator of first fraction.

You should allow entry of different numerators and denominators for both fractions.



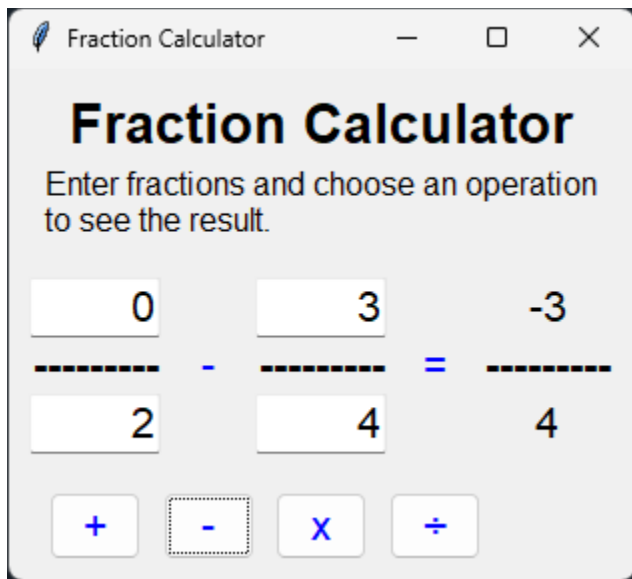
Press + button.

Operations should give the correct result when selected.



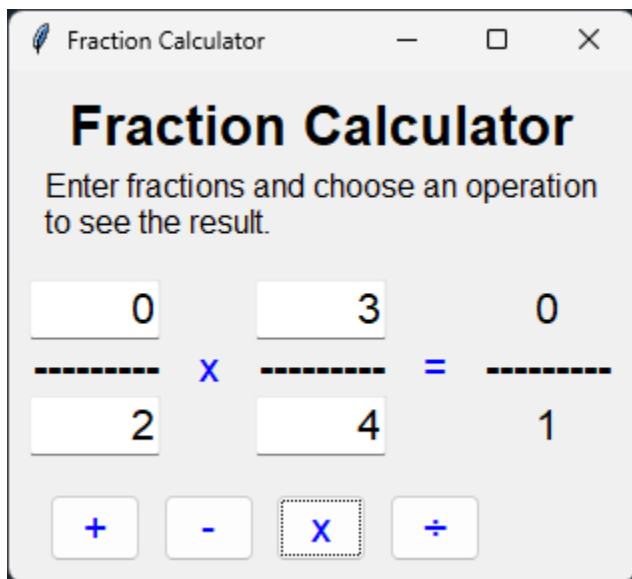
Press - button.

Operations should give the correct result when selected.



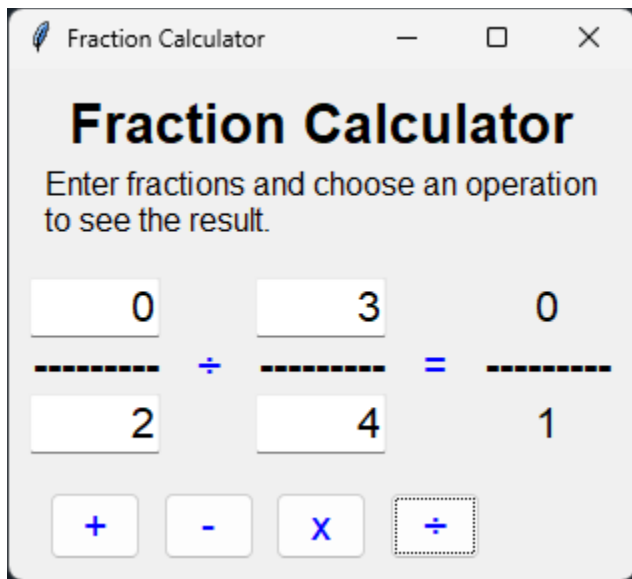
Press x button.

Operations should give the correct result when selected.



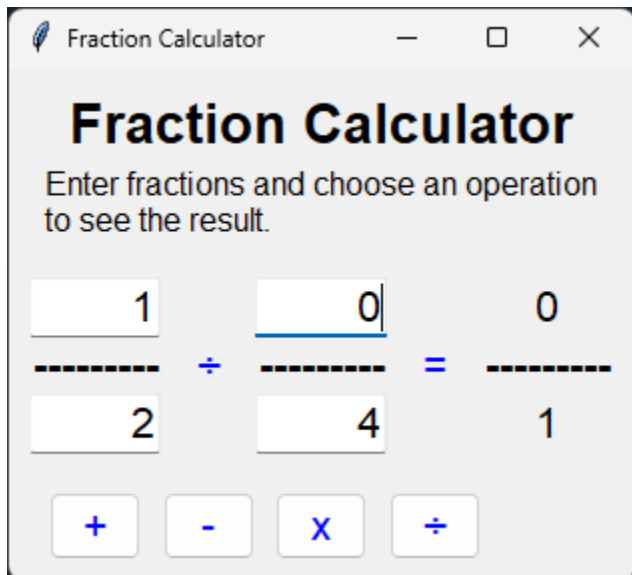
Press ÷ button.

Operations should give the correct result when selected.



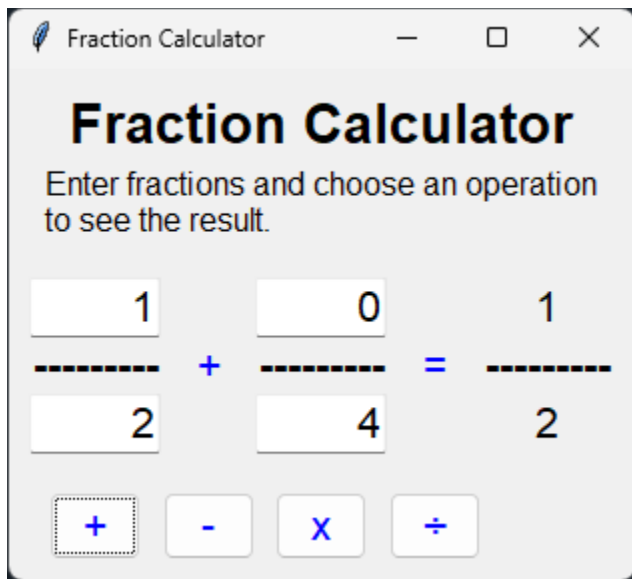
Change 0 back to 1 in numerator of first fraction. Change 3 to 0 in numerator of second fraction.

You can change the numerator and/or denominator of either or both fractions.



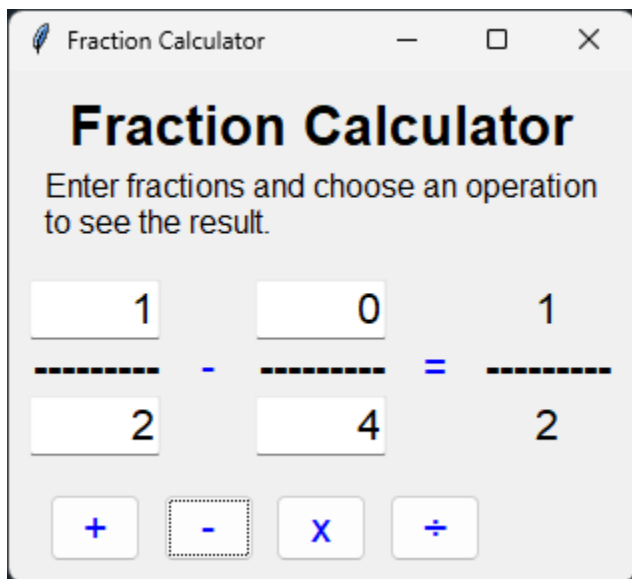
Press + button.

Operations should give the correct result when selected.



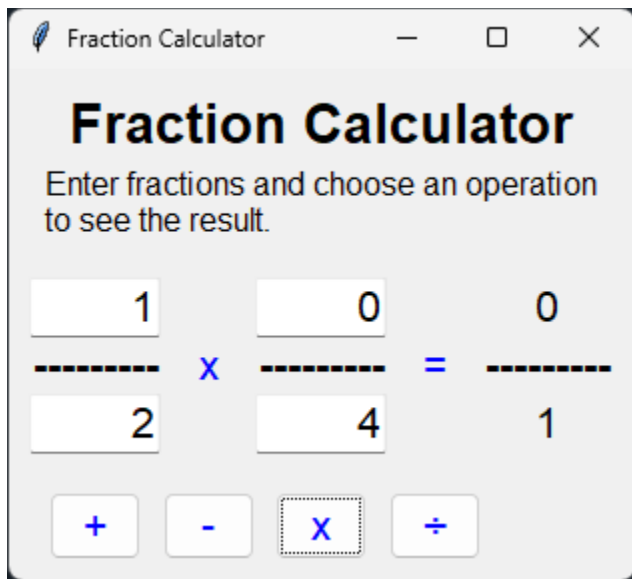
Press - button.

Operations should give the correct result when selected.



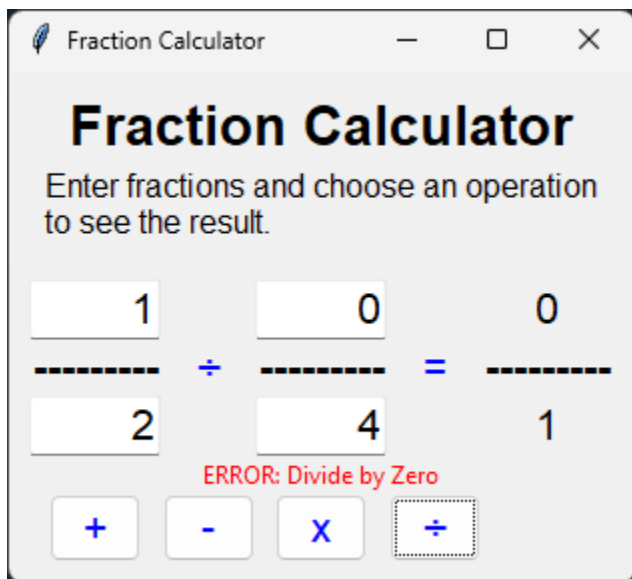
Press x button.

Operations should give the correct result when selected.



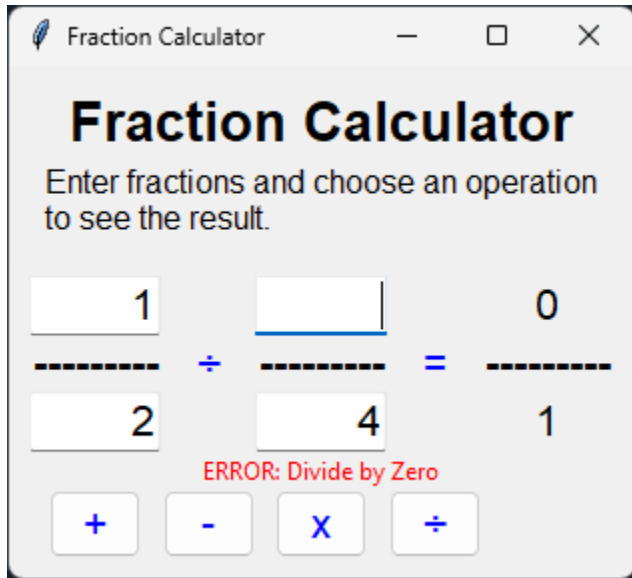
Press ÷ button.

Errors should be shown when operations cannot be carried out.



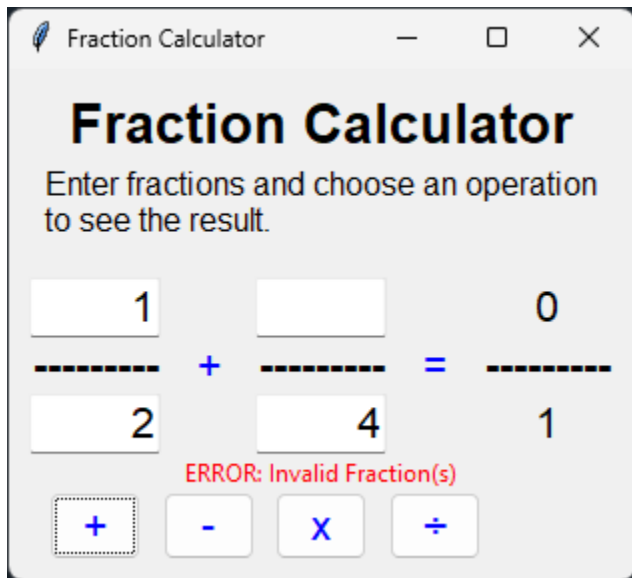
Delete 0 in numerator of second fraction, leaving entry box empty.

The entry widgets allow entry of any text, not just numbers. This includes no entry. The prior error message is displayed until either an operation is successfully carried out or another error occurs.



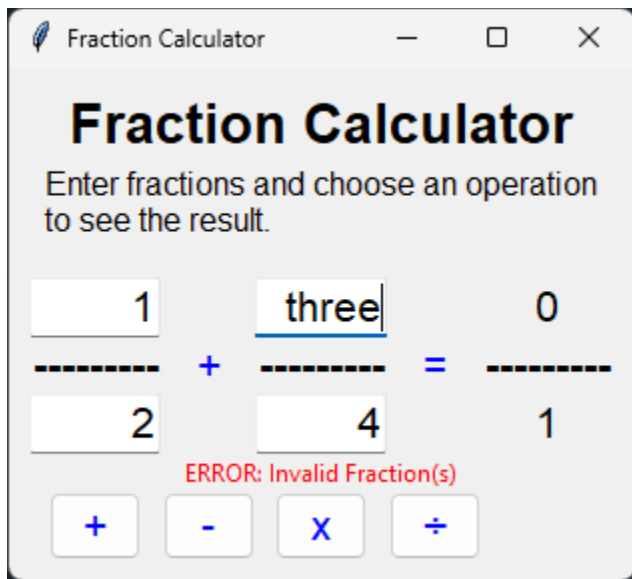
Press + button.

Errors should be shown when operations cannot be carried out.



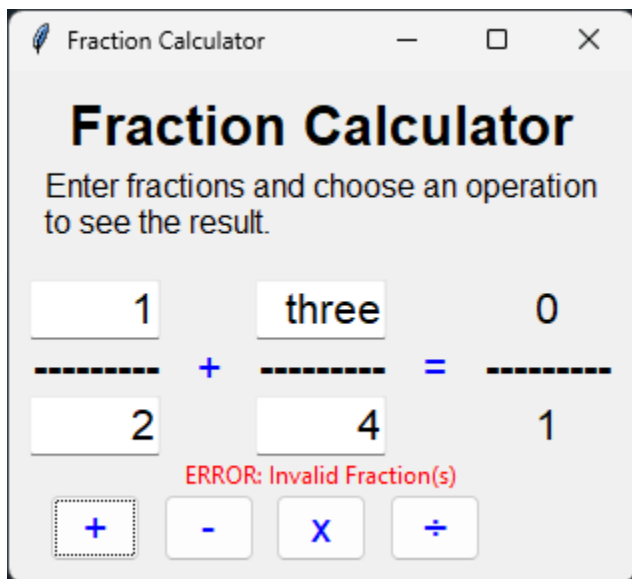
Enter the word three in numerator of second fraction.

The entry widgets allow entry of any text, not just numbers.



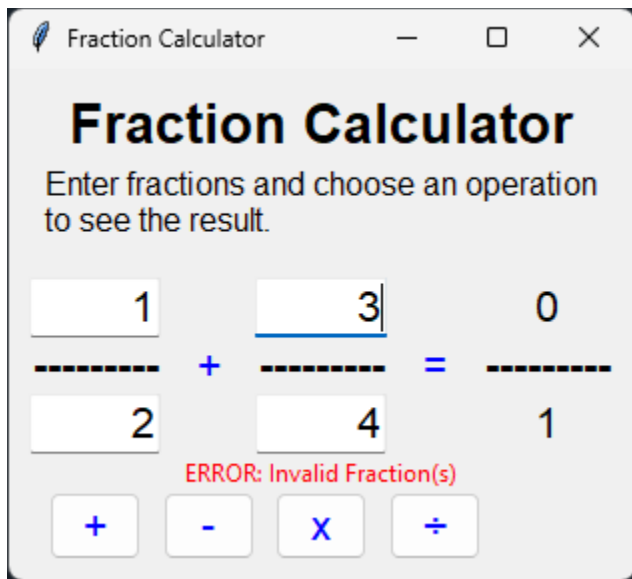
Press + button.

Errors should be shown when operations cannot be carried out.



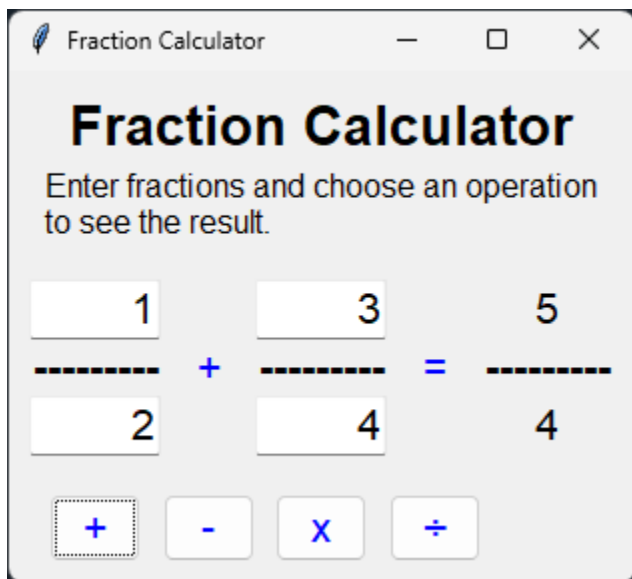
Enter 3 in numerator of second fraction.

The error message remains until another operation is attempted.



Press + button.

When an operation is carried out successfully, the prior error message is removed.



Helpful hints:

Use the example file that I provided. Read it, run it, use it as a guide.

Work incrementally. Build and test incrementally, starting with easier aspects. Add functionality and/or change display elements incrementally, testing with each change.

As mentioned earlier, there are multiple aspects to this assignment. Ideally you will successfully address all aspects. If you get stuck, it may be helpful to use the following prioritization which lists most important first and less important afterwards:

- correct functionality,
- correct display,
- use of prior functions or classes.

Interpret this as follows:

- Most important is that you have a calculator that functions as described. I.e., it takes input, and computes and shows results correctly.
- Next most important is that it looks like what was specified. Once the functionality is correct, you can fine-tune the display.
- Finally, make use of your ExceptionalFractions.py or ClassyFraction.py file.

Having a working calculator that doesn't look quite right is better than a non-functioning calculator that looks right. If you still have problems with your ExceptionalFractions.py or ClassyFraction.py file, you should fix them. You could also try to implement the necessary functionality without them. You will lose points in doing so but you might get your calculator to function and look right separately.

Another approach is to get the layout and display correct, and then implement the corresponding functionality. A risk with this approach is that you don't end up with a functioning calculator, particularly if you take too long fine-tuning the display.

Don't just check for proper calculation. Check for invalid inputs and other things that could go wrong. Try to break your code and implement fixes so it won't break when we test it. Generating known exceptions for invalid inputs is part of this, as well as gracefully displaying errors in the GUI as described rather than allow the program to crash. Using your prior work properly should help with this.

Upload FractionCalculator.py to Moodle as your submission for this assignment. Also upload ExceptionalFractions.py or ClassyFractions.py, depending on which you used for your fraction calculator.

EXTENSIONS

You will not submit the answers to these extensions as part of your assignment submission. You should work through and understand them to learn more and practice. You may be held responsible for the content within, so you really should work through them ...

From me:

1. Make a calculator that incorporates all fraction functionality implemented in ExceptionalFractions.py or ClassyFraction.py.
2. What did/does it take to implement functionality corresponding to unary operators, non-fraction data (integer exponents), etc. for the calculator described in #1?